A Literature Review of Deep Learning Applications in NLP:

Text-to-SQL Parsing and Reasoning, with a Proposed Semester Project

Focus Paper: PARSQL — Enhancing Text-to-SQL through SQL Parsing and Reasoning (Dai et al., ACL Findings 2025)

Darshi Shah (C037, 60004230219)

Rishi Shah (C128, 60004230245)

Shubh Vora (C145, 60004230013)

1. Introduction

Text-to-SQL — the task of translating a natural-language question into an executable SQL query over a relational database — is one of the most practically important applications of deep learning to Natural Language Processing (NLP). It sits at the intersection of semantic parsing, program synthesis, and grounded language understanding: a system must interpret an ambiguous, informally phrased question, link its entities to a specific database schema, and produce a syntactically and semantically correct formal query. Since 2023, the field has been reshaped by large language models (LLMs), which brought large accuracy gains but also new failure modes — hallucinated columns, incorrect aggregation logic, brittle multi-table joins — and new costs, since the strongest systems often depend on very large parameter counts and expensive multi-step inference pipelines.

Our chosen topic is PARSQL: Enhancing Text-to-SQL through SQL Parsing and Reasoning (Dai, Yang, Hao, & Chao, 2025), published in Findings of the Association for Computational Linguistics: ACL 2025. PARSQL asks a pointed question that much of the field has skipped over while chasing frontier-scale LLMs: can small language models (SLMs) — cheaper, faster, and more deployable — be made competitive on text-to-SQL if they are given an explicit, structured reasoning process over SQL, instead of being asked to generate SQL as a single opaque decoding step? PARSQL answers this by parsing gold SQL queries into constituent parts to build training signal for both SQL generation and an auxiliary "reasoning" task, and by using a lightweight, similarity-based strategy to pick the best candidate SQL at inference time. The result is a 3-billion-parameter model that rivals much larger 7-billion-parameter systems on standard benchmarks.

This report proceeds in three parts. Section 2 reviews eight additional 2025 papers on deep learning applied to text-to-SQL, situating PARSQL within the broader research landscape. Section 3 examines PARSQL itself in depth — its architecture, training objective, experimental results, and limitations — since it is the anchor topic of this report. Section 4 proposes a semester-long case study that extends PARSQL's method to a genuinely under-served setting: code-mixed Hindi–English ("Hinglish") text-to-SQL, complete with a dataset plan, target languages, timeline, and evaluation protocol. Section 5 concludes.

2. Literature Review

The following nine papers, all published in 2025, address text-to-SQL as a reasoning-intensive deep learning problem for NLP. For each paper we give its name, a short introduction to the problem it tackles, and a combined summary of its methodology, key findings, and limitations.

2.1 PARSQL: Enhancing Text-to-SQL through SQL Parsing and Reasoning

This is the focus paper of the present report. It targets small language models (SLMs) rather than very large LLMs, since SLMs are essential for cost- and latency-sensitive real-world deployment but typically under-perform on text-to-SQL because of the semantic gap between flexible natural language and rigid, structured SQL — for instance, producing MIN(population) when the question actually implies MAX(population).

PARSQL's core contribution is PARSer, an SQL parser that walks the abstract syntax tree of a gold SQL query to extract constraints and split it into sub-SQL fragments for data augmentation, and to generate

step-by-step reasoning explanations using both rule-based templates and an LLM. A new text-to-reason task is trained jointly with text-to-SQL in a multi-task setup, and at inference time an efficient SQL-selection strategy computes similarity between candidate SQLs and their reasoning text to choose the final, corrected query without expensive execution-based re-ranking. The method outperforms same-size baselines on BIRD and Spider, with PARSQL-3B reaching 56.98% execution accuracy on BIRD and rivaling much larger 7B models. Its main limitations are that the rule-based parser and reasoning generation are validated only on clean, English benchmarks, and reasoning-trace quality depends on the teacher LLM used to help generate it, so noise can propagate into training data.

2.2 CHASE-SQL: Multi-Path Reasoning and Preference-Optimized Candidate Selection in Text-to-SQL

This paper tackles the inconsistency of single-pass LLM decoding for SQL generation by asking how multiple diverse reasoning paths can be generated and then reliably ranked to select the best final query.

CHASE-SQL generates diverse SQL candidates via divide-and-conquer decomposition of complex questions, chain-of-thought reasoning grounded in the query's execution plan, and instance-aware synthetic few-shot example selection, then uses a separately fine-tuned binary preference model to rank candidates through pairwise rather than pointwise comparison. This combination achieved state-of-the-art execution accuracy of 73.0% on the BIRD test set at submission time. Its central limitation is cost: generating multiple reasoning paths per question and running a dedicated selection model multiplies inference expense, conflicting with real-world latency and cost budgets — a concern that later papers such as EllieSQL and PARSQL explicitly try to resolve.

2.3 Reasoning-SQL: Reinforcement Learning with SQL-Tailored Partial Rewards for Reasoning-Enhanced Text-to-SQL

Motivated by reasoning models such as OpenAI o1 and DeepSeek-R1, this paper asks whether reinforcement learning (RL) with task-specific rewards can teach an LLM intrinsic SQL reasoning, rather than relying on handcrafted chain-of-thought templates that introduce inductive bias.

The authors design four partial, dense rewards tailored to text-to-SQL — schema-linking, AI-feedback, n-gram similarity, and syntax-check — to counter the sparsity of using execution accuracy alone, and train with Group Relative Policy Optimization (GRPO). RL training with these rewards consistently outperforms supervised fine-tuning across model sizes and generalizes better to unseen databases, showing that reward design, not just more supervised data, is a key lever for text-to-SQL reasoning. The main limitation is that reward engineering is itself a manual, domain-specific effort requiring four components to be tuned and balanced, and the paper does not demonstrate that this reward recipe transfers unchanged to other query languages or multilingual questions.

2.4 ExCoT: Optimizing Reasoning for Text-to-SQL with Execution Feedback

This paper investigates why naive chain-of-thought (CoT) prompting yields only minimal text-to-SQL gains, and why Direct Preference Optimization (DPO) without CoT also produces marginal improvements, then asks how the two can be combined without human-annotated preference data.

ExCoT iteratively fine-tunes open-source LLMs by alternating CoT-reasoning generation with both off-policy and on-policy DPO, using execution accuracy as the sole preference signal so that no separate reward model or human annotation is required. This raises execution accuracy from 57.37% to 68.51% on the BIRD dev set and from 78.81% to 86.59% on the Spider test set for LLaMA-3 70B, with similar gains for Qwen-2.5-Coder. Its limitations are that execution-only feedback cannot distinguish a query that is accidentally correct from one that is correct for the right reasons, leaving reasoning faithfulness unverified, and the method is demonstrated only at large (70B-class) model scale.

2.5 EllieSQL: Cost-Efficient Text-to-SQL with Complexity-Aware Routing

This paper names the "elephant in the room" of leaderboard-driven text-to-SQL research: state-of-the-art pipelines reach high accuracy but at unsustainable computational cost, because every query — simple or complex — is routed through the same expensive, multi-step reasoning pipeline.

EllieSQL introduces a complexity-aware router that estimates a query's difficulty after schema linking and then dynamically sends it to one of several tiers of SQL-generation pipelines, reserving the most expensive multi-agent or CoT-heavy methods for genuinely hard queries; a new Token Elasticity of Performance (TEP) metric quantifies accuracy gained per unit of additional token spend. This preserves most of the accuracy of always using the most expensive pipeline while substantially cutting token cost, showing accuracy and efficiency are not strictly coupled. The key limitation is that the router itself must be trained on labeled complexity judgments that may not generalize across domains, and misrouting a hard query as easy directly damages accuracy on exactly the cases that matter most.

2.6 Reward-SQL: Boosting Text-to-SQL via Stepwise Execution-Aware Reasoning and Process-Supervised Rewards

This paper studies how to incorporate Process Reward Models (PRMs) — which score intermediate reasoning steps rather than only the final answer — into text-to-SQL training, since naive PRM use can distort the reasoning trajectory and produce worse SQL than using no PRM at all.

The authors introduce Chain-of-CTEs (CoCTE): the model is trained to decompose a target SQL query into a sequence of Common Table Expressions, each independently executable and verifiable, giving a natural unit for process supervision, and training follows a "cold start, then PRM supervision" curriculum. This stepwise, execution-aware approach substantially reduces specific error types — table-selection errors by 42.6%, column-selection errors by 27.2%, and GROUP BY errors by 82.4% — by turning a single sparse execution signal into dense, per-step feedback. Its limitations are that the CoCTE decomposition assumes a CTE-capable SQL dialect and independently meaningful intermediate steps, which may not hold for all dialects or deeply nested queries, and building the PRM requires additional step-level annotation or synthesis beyond the base dataset.

2.7 ReEx-SQL: Reasoning with Execution-Aware Reinforcement Learning for Text-to-SQL

Prior work treats database execution feedback as a post-hoc signal used only to correct or re-rank a finished SQL query. This paper, from a subset of the same research group as PARSQL, asks whether execution feedback can instead be interleaved directly into the reasoning process so errors are caught while reasoning is still in progress.

ReEx-SQL trains with a structured prompt format using explicit markup tokens (<think>, <intermediate_sql>, <result>, <final_sql>), executing exploratory SQL fragments mid-reasoning and feeding results back into the next step, guided by a composite RL reward with an explicit exploration bonus and a tree-based decoding strategy for branching reasoning paths. It reaches 88.8% execution accuracy on Spider and 64.9% on BIRD at 7B scale, improving over a standard one-pass reasoning baseline by 2.7 and 2.6 points respectively. Its limitations include requiring live, low-latency database access during both training and inference, which raises deployment cost, and the authors themselves note the method has only been tested in English, with applicability to other domains or languages left unexplored.

2.8 SQL-R1: Training Natural Language to SQL Reasoning Model by Reinforcement Learning

This paper argues that supervised fine-tuning (SFT) alone limits the adaptability and interpretability of NL2SQL models in high-stakes domains such as finance and healthcare, where multi-table joins and nested queries are common, and asks whether RL-only training can produce a more robust reasoning model.

SQL-R1 trains directly with RL using a specialized NL2SQL reward function, studying how a small "cold start" supervised phase and limited synthetic data affect downstream RL effectiveness, aiming to reduce dependence on heavy human annotation. Using only a 7B base model and a small amount of synthetic data, it achieves 88.6% execution accuracy on Spider and roughly 66–67% on BIRD, competitive with far more heavily supervised or larger systems. The paper's own future-work discussion flags interpretability and larger-scale multi-table reasoning as open problems, and like most of this literature, evaluation is restricted to English-language Spider/BIRD-style benchmarks.

2.9 Gen-SQL: Efficient Text-to-SQL by Bridging Natural Language Question and Database Schema with Pseudo-Schema

For large, real-world databases, retrieving the small relevant subset of tables and columns before SQL generation is critical, but existing retrieve-then-generate pipelines force a trade-off: cheap embedding-based retrievers have low accuracy, while accurate LLM-based retrievers are too expensive for broad deployment.

Gen-SQL proposes a generate-ground-regenerate framework: the LLM first generates a plausible "pseudo-schema" directly from the question using its own prior knowledge, this pseudo-schema is then grounded against the real schema via an embedding-based retriever now guided by richer context, and the SQL is finally (re)generated against the grounded schema, optionally iterating the loop. By using LLM

prior knowledge to sharpen a cheap retriever rather than replacing it, Gen-SQL narrows the accuracy gap to expensive LLM-based retrieval while keeping cost low, across several real-world-scale datasets. Its main limitation is that pseudo-schema generation depends on the LLM already having reasonable prior knowledge of plausible schema structure for the domain, so for narrow, specialized, or non-English domains where this prior is weak, the generation step may itself hallucinate table or column names that mislead grounding.

2.10 Synthesis

Three cross-cutting patterns emerge from these nine papers. First, every paper after PARSQL converges on some form of explicit, checkable intermediate reasoning — sub-SQL decomposition, chain-of-CTEs, execution-interleaved reasoning, or multi-path generation — rather than a single opaque decode, echoing PARSQL's own text-to-reason multi-task design. Second, reinforcement learning with execution-grounded or execution-adjacent rewards (Reasoning-SQL, ExCoT, Reward-SQL, ReEx-SQL, SQL-R1) has become the dominant training paradigm for 2025-era text-to-SQL, replacing pure supervised fine-tuning. Third, and most relevant to the project proposed in Section 4, almost every limitation identified above is an English-only limitation: none of the nine papers evaluates a non-English or code-mixed benchmark, despite text-to-SQL's obvious value for non-English-speaking users of enterprise and government databases.

3. Our Topic in Focus: PARSQL — Enhancing Text-to-SQL through SQL Parsing and Reasoning

Having placed PARSQL within the broader 2025 text-to-SQL literature, this section examines it in depth, since it is the anchor paper for this report and for the semester project proposed in Section 4.

3.1 Problem and Motivation

Large language models have driven rapid progress on text-to-SQL, but PARSQL's authors observe that most of this progress has come from scaling model size, which is at odds with the practical need for small, cheap, low-latency models in production database systems. Small language models (SLMs) are attractive for exactly this reason, yet they consistently underperform larger LLMs on text-to-SQL because of a semantic gap: natural language questions are flexible and often underspecified, while SQL is rigid and formal, so an SLM without strong reasoning support tends to make systematic errors such as selecting the wrong aggregation function or misidentifying join conditions. PARSQL's motivating question is whether this gap can be closed not by adding parameters, but by giving the model explicit structural and reasoning supervision derived directly from the SQL it is trying to learn to generate.

3.2 System Architecture: PARSer, Sub-SQL Augmentation, and Reasoning Generation

The central technical component of PARSQL is PARSer, a purpose-built SQL parser that operates on the abstract syntax tree (AST) of each gold SQL query in the training data. PARSer serves two functions. First, it extracts the constraints embedded in a query's clauses (selection conditions, joins, groupings, aggregations) and uses them to systematically split a complex query into smaller sub-SQL fragments, which are used to augment the training data with additional, simpler (question, SQL) pairs that still reflect real query structure — addressing the common problem of complex queries being under-represented relative to their difficulty. Second, PARSer produces step-by-step natural-language

explanations of what a query does and why, combining rule-based templates (for constructs whose semantics are unambiguous from the AST) with LLM-based generation (for constructs that need more natural phrasing), yielding a "reason" text paired with every SQL query.

3.3 Multi-Task Learning Objective

PARSQL formalizes the reasoning text as a new task, text-to-reason, and trains it jointly with the standard text-to-SQL task in a multi-task learning setup. This means the same model backbone learns to map a question to its SQL query and, separately, to map a question to a natural-language explanation of the reasoning needed to answer it. The intuition is that the reasoning task forces the model to represent the intermediate semantic steps (which tables are needed, what the aggregation should be, what conditions apply) explicitly, rather than leaving them implicit inside a single SQL-generation decoder — the same intuition later formalized more explicitly by Reward-SQL's Chain-of-CTEs and ReEx-SQL's execution-interleaved reasoning (Section 2).

3.4 Inference-Time SQL Selection Strategy

At inference time, PARSQL generates multiple candidate SQL queries together with their associated reasoning texts. Rather than relying on an expensive execution-based re-ranking step (running each candidate against the database and comparing results, as several other papers in Section 2 do), PARSQL computes a direct similarity score between each candidate SQL and its paired reasoning text and selects the pair with the strongest alignment. This is then used for a lightweight post-correction step. This design choice is what gives PARSQL its efficiency advantage over approaches such as CHASE-SQL, which depends on a separately trained, fine-tuned preference model to rank candidates.

3.5 Experimental Results

PARSQL is evaluated on the two standard cross-domain text-to-SQL benchmarks, Spider and BIRD, and is shown to outperform other models of the same parameter count on both. The headline result is that PARSQL-3B achieves 56.98% execution accuracy on BIRD, a benchmark of complex, real-world, domain-specific databases, rivaling 7-billion-parameter models while using well under half the parameters. This supports the paper's central claim: with the right parsing-and-reasoning supervision, a small model can substitute, at least partially, for raw scale.

3.6 Strengths and Position Relative to Related Work

Relative to the eight other 2025 papers reviewed in Section 2, PARSQL's distinguishing strength is efficiency at the point of both training-data construction and inference: PARSer's rule-based sub-SQL splitting is comparatively cheap to run at scale, and the similarity-based selection strategy avoids the repeated database execution or separately trained reward/selection model that CHASE-SQL, Reward-SQL, Reasoning-SQL, and ReEx-SQL variously require. In this sense, PARSQL sits at a distinctive point in the design space between purely execution-driven RL methods and purely prompting-based methods, using a static, parser-derived signal that does not need live database access to train or run.

3.7 Limitations and Research Gaps

PARSQL's evaluation, like the rest of the literature reviewed in Section 2, is confined to English-language questions over the Spider and BIRD benchmarks. The rule-based half of PARSer's reasoning generation is designed around SQL constructs that are unambiguous in a formal, English-adjacent sense, and its LLM-

assisted half inherits whatever errors or stylistic biases the teacher LLM has. Because the paper does not test performance when the natural-language side is a low-resource, morphologically rich, or code-mixed language, it leaves open exactly the gap this report's proposed project (Section 4) sets out to investigate: how much of PARSQL's small-model efficiency advantage survives when the input language itself becomes harder.

4. Proposed Semester Project: PARSQL-Hinglish

4.1 Motivation

Section 3.7 identified a clear and recurring gap: PARSQL and all eight related 2025 papers reviewed in Section 2 are built and evaluated exclusively in English, even though PARSQL's specific contribution — recovering strong accuracy from small, deployable models via explicit SQL parsing and reasoning supervision — is arguably even more valuable in a multilingual, resource-constrained setting than in an English-only, large-model setting. India's enterprise and government data systems are frequently queried by users who naturally mix Hindi and English (Hinglish) in speech and typed text. A project that adapts PARSQL's architecture to this setting has both clear intellectual grounding in an existing, code-available method and clear real-world relevance.

4.2 Objectives

- Reproduce the PARSQL baseline on English Spider/BIRD to establish a verified starting point.
- Construct a Hindi and Hinglish parallel extension of a Spider/BIRD subset, following the multi-round translation-and-validation methodology introduced by MultiSpider, since the same weaknesses MultiSpider identifies (unnatural translation, incomplete schema translation) must be avoided for a usable Hinglish resource.
- Adapt PARSQL's PARSer and multi-task (text-to-SQL + text-to-reason) training objective so it operates correctly when the natural-language side is Hindi or code-mixed, while the SQL side and its structural parsing remain unchanged.
- Empirically measure how much of PARSQL's small-model advantage survives the move from English to Hindi/Hinglish, and which specific error types (schema linking vs. aggregation vs. join logic) are most affected by the language shift.

4.3 Proposed Methodology

The project keeps PARSQL's architecture as the backbone. PARSer will continue to operate on the SQL abstract syntax tree exactly as in the original paper (SQL is language-independent), so sub-SQL extraction for data augmentation transfers unchanged. The text-to-reason auxiliary task, however, must be regenerated: rule-based reasoning templates need Hindi/Hinglish surface forms, and the LLM-based reasoning generation step will be re-run using a multilingual-capable LLM prompted to produce reasoning text in the same language/register as the input question. Training follows PARSQL's multi-task learning setup (joint text-to-SQL and text-to-reason objectives) on three separate splits — English, Hindi, and Hinglish — so that language-specific degradation can be isolated cleanly rather than confounded. At inference, PARSQL's similarity-based SQL-selection strategy is retained as the low-cost alternative to execution-based re-ranking, since this is the component most directly responsible for PARSQL's efficiency advantage over CHASE-SQL-style multi-agent selection.

4.4 Dataset Availability and Specifications

Table 1 lists the datasets required for this project, all either already public or constructible with a well-precedented methodology within the semester timeline.

Dataset	Size	Format	Source / Availability	Role in the Project
Spider 1.0	10,181 NL questions; 5,693 unique SQL queries; 200 databases (138 domains)	JSON (question, SQL, db_id, schema) + SQLite DB files	Yale LILY Lab; public, free for research. https://yale-lily.github.io/spider (Yu et al., 2018)	English source pairs for building the Hinglish parallel corpus; base cross-domain evaluation
BIRD (BIg Bench for LaRge-scale Database Grounded Text-to-SQL)	12,751 NL-SQL pairs; 95 databases; 33.4 GB of real database content across 37 domains	JSON (question, SQL, evidence, db_id) + SQLite DB files	BIRD-Bench; public, free for research. https://bird-bench.github.io/ (Li et al., 2023/2024)	Realistic, noisy, execution-heavy source questions to stress-test Hinglish reasoning on messy real-world schemas
MultiSpider	9,691 questions over 166 databases, translated across 7 languages (En, De, Fr, Es, Ja, Zh, Vi)	JSON, parallel to Spider format, per-language question files	Public GitHub + HuggingFace release. https://github.com/longxudou/multispider (Dou et al., 2023)	Methodological template: reused translation-and-validation pipeline for building the new Hindi/Hinglish split
IndicDB (emerging Indic text-to-SQL benchmark)	Multi-fact, star/snowflake-schema questions across 6 Indic languages incl. Hindi, Marathi, Bengali, Tamil, Telugu, and Hinglish	JSON, schema + gold SQL per language	arXiv preprint (2026); availability to be confirmed with authors before the semester begins. https://arxiv.org/abs/2604.13686	Candidate external benchmark for cross-lingual generalization testing if released data can be obtained in time
Project-built: Hinglish-Spider-Aug	Planned ~2,000-3,000 question subset (stratified sample of Spider + BIRD dev questions)	JSON, extending Spider/BIRD schema with hi/hi-en question fields + PARSer-derived sub-SQL/reason fields	Newly created by the project team using LLM-assisted translation + native-speaker verification (following MultiSpider's quality-control protocol)	Primary training/evaluation resource, produced in Phase 1 of the timeline below

Table 1. Dataset inventory, specifications, and role in the proposed project.

4.5 Target Languages

The project targets three conditions for controlled comparison: (1) English, as the reproduction baseline directly comparable to the original PARSQL paper; (2) Hindi (formal, Devanagari-script) as a genuinely low-resource condition relative to Spider/BIRD's English-only training data; and (3) Hinglish (Romanized, code-mixed Hindi–English), which reflects how Indian users actually type natural-language queries in

practice and is the primary novel contribution of the project, since no paper reviewed in Section 2 evaluates a code-mixed setting.

4.6 Semester Timeline

Weeks	Phase	Activities	Deliverable
1-2	Setup & literature grounding	Reproduce PARSQL baseline on Spider/BIRD (English) using released code; confirm the SQL parser (PARSer) works on both benchmarks' SQL dialects.	Working PARSQL baseline + reproduction report
3-5	Hinglish dataset construction	Sample ~2,500 Spider/BIRD dev questions; generate Hindi and Hinglish (code-mixed) translations via LLM-assisted translation; run MultiSpider-style multi-round validation with native Hindi speakers to fix mistranslations and schema-term mismatches.	Hinglish-Spider-Aug v1 dataset with quality report
6-7	PARSer adaptation	Extend PARSQL's SQL parser/reasoning generator so sub-SQL splitting and rule-based reasoning remain language-agnostic (operate on the SQL side only), while the text-to-reason task is retrained on Hindi/Hinglish question-reason pairs.	Adapted PARSer + text-to-reason training set
8-10	Model training	Fine-tune 1-3B-parameter open SLMs (e.g., a Qwen2.5-Coder or CodeS-class model) with PARSQL's multi-task (text-to-SQL + text-to-reason) objective on the Hinglish dataset; establish English-only, Hindi-only, and Hinglish training/test splits.	Trained checkpoints for 3 language conditions
11-12	Evaluation	Measure execution accuracy and exact-match accuracy per language condition on held-out Spider/BIRD-derived test sets; run PARSQL's SQL-selection strategy; conduct an error-category analysis	Results tables + error analysis

		(schema-linking vs. aggregation vs. join errors) as in Reward-SQL's methodology.	
13-14	Ablations & write-up	Ablate the text-to-reason auxiliary task and the sub-SQL augmentation separately to isolate their contribution in the multilingual setting; compare against a non-PARSQL baseline (plain SFT) under the same Hinglish data.	Ablation study
15-16	Final report & demo	Compile final report, prepare a small interactive demo (natural-language-to-SQL over one sample database in Hindi/Hinglish), and submit all code/data with documentation.	Final report, demo, and submission package

Table 2. Proposed 16-week semester timeline. Adjust week numbers to the institution's academic calendar and submit the final report by the deadline specified in the course syllabus.

4.7 Evaluation Plan

- Primary metric: execution accuracy (matching PARSQL, CHASE-SQL, ExCoT, and the RL-based papers reviewed in Section 2), computed separately for English, Hindi, and Hinglish test splits.
- Secondary metric: exact-match / component-match accuracy on SQL clauses (SELECT, WHERE, GROUP BY, JOIN), enabling the error-category breakdown used by Reward-SQL to be replicated in the multilingual setting.
- Ablations: (a) PARSQL full model vs. PARSQL without the text-to-reason auxiliary task, (b) PARSQL vs. a plain supervised-fine-tuning baseline with no sub-SQL augmentation, both repeated across the three language conditions.

4.8 Expected Outcomes and Limitations

Expected outcome: a quantified picture of how much of PARSQL's small-model efficiency advantage transfers to Hindi/Hinglish, and a reusable Hinglish-Spider-Aug dataset that can seed further multilingual text-to-SQL work. Anticipated limitations, to be stated transparently in the final report: (1) the translated Hinglish dataset, built via LLM-assisted translation with native-speaker verification, will be smaller (~2,000-3,000 questions) than the full English Spider/BIRD sets and may not capture every dialectal variant of Hinglish; (2) as with ReEx-SQL and Reward-SQL, any execution-based verification step requires functioning SQLite copies of the Spider/BIRD databases, which is feasible but adds engineering overhead; (3) if the emerging IndicDB benchmark (Section 4.4) is not accessible in time, the project will rely solely on the self-built Hinglish-Spider-Aug set, which somewhat narrows the domains covered.

5. Conclusion

The nine papers reviewed in Section 2 trace a clear trajectory in 2025 text-to-SQL research: from PARSQL's SQL-parser-driven reasoning supervision for small models, through multi-path candidate generation and selection (CHASE-SQL), to reinforcement-learning-based reasoning with increasingly fine-grained, execution-grounded rewards (Reasoning-SQL, ExCoT, Reward-SQL, ReEx-SQL, SQL-R1), and finally to efficiency and retrieval-oriented refinements (EllieSQL, Gen-SQL). Section 3's closer look at PARSQL showed that its parsing-and-reasoning architecture is efficient, well validated on English benchmarks, and architecturally well suited to extension, since its SQL-side parser is language independent by construction. Despite this rapid progress, every one of these systems is built and tested only in English. The semester project proposed in Section 4 directly addresses this gap by extending PARSQL to Hindi and Hinglish, using a combination of the well-established Spider and BIRD benchmarks, the MultiSpider translation methodology, and a newly constructed parallel dataset, with a concrete 16-week timeline and evaluation plan for producing a rigorous, reproducible semester deliverable.

References

Dai, Y., Yang, H., Hao, M., & Chao, P. (2025). PARSQL: Enhancing text-to-SQL through SQL parsing and reasoning. In Findings of the Association for Computational Linguistics: ACL 2025 (pp. 661–681). Association for Computational Linguistics. https://aclanthology.org/2025.findings-acl.37/

Dai, Y., Xie, W., Zhuang, X., Yang, T., Yang, Y., Yang, H., Zhao, Y., Chao, P., & Jiang, W. (2025). ReEx-SQL: Reasoning with execution-aware reinforcement learning for text-to-SQL. arXiv preprint arXiv:2505.12768. https://arxiv.org/abs/2505.12768

Dou, L., Gao, Y., Pan, M., Wang, D., Che, W., Zhan, D., & Lou, J.-G. (2023). MultiSpider: Towards benchmarking multilingual text-to-SQL semantic parsing. In Proceedings of the AAAI Conference on Artificial Intelligence. https://ojs.aaai.org/index.php/AAAI/article/view/26499

Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C.-C., Huang, F., Cheng, R., & Li, Y. (2024). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs (BIRD). Advances in Neural Information Processing Systems, 36. https://bird-bench.github.io/

Ma, P., Zhuang, X., Xu, C., Jiang, X., Chen, R., & Guo, J. (2025). SQL-R1: Training natural language to SQL reasoning model by reinforcement learning. arXiv preprint arXiv:2504.08600. https://arxiv.org/abs/2504.08600

Pourreza, M., Li, H., Sun, R., Chung, Y., Talaei, S., Kakkar, G. T., Gan, Y., Saberi, A., Ozcan, F., & Arik, S. O. (2025). CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL. In Proceedings of the International Conference on Learning Representations (ICLR 2025). https://openreview.net/forum?id=CvGqMD5OtX

Pourreza, M., Talaei, S., Sun, R., Wan, X., Li, H., Mirhoseini, A., Saberi, A., & Arik, S. O. (2025). Reasoning-SQL: Reinforcement learning with SQL tailored partial rewards for reasoning-enhanced text-to-SQL. arXiv preprint arXiv:2503.23157. https://arxiv.org/abs/2503.23157

Shi, J., Xu, B., Liang, J., Xiao, Y., Chen, J., Xie, C., Wang, P., & Wang, W. (2025). Gen-SQL: Efficient text-to-SQL by bridging natural language question and database schema with pseudo-schema. In Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025) (pp. 3794–3807). Association for Computational Linguistics. https://aclanthology.org/2025.coling-main.256/

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 3911–3921). https://yale-lily.github.io/spider

Zhai, B., Xu, C., He, Y., & Yao, Z. (2025). ExCoT: Optimizing reasoning for text-to-SQL with execution feedback. arXiv preprint arXiv:2503.19988. https://arxiv.org/abs/2503.19988

Zhang, Y., Fan, M., Fan, J., Yi, M., Luo, Y., Tan, J., & Li, G. (2025). Reward-SQL: Boosting text-to-SQL via stepwise execution-aware reasoning and process-supervised rewards. arXiv preprint arXiv:2505.04671. https://arxiv.org/abs/2505.04671

Zhu, Y., Jiang, R., Li, B., Tang, N., & Luo, Y. (2025). EllieSQL: Cost-efficient text-to-SQL with complexity-aware routing. In Proceedings of the Conference on Language Modeling (COLM 2025); arXiv preprint arXiv:2503.22402. https://arxiv.org/abs/2503.22402